

Spring data jpa handson

Hands-on 1: Spring Data JPA – Quick Example

Aim

To create a Spring Boot application using Spring Data JPA and MySQL, map a database table to a Java entity, and retrieve country records using JpaRepository.

Software Requirements

- Java 8 or above
- MySQL Server 8.0
- MySQL Workbench 8
- Eclipse IDE for Enterprise Java Developers
- Maven 3.6.2
- Spring Boot

Step 2: Create Database and Table

Execute the following SQL commands.

```
CREATE SCHEMA ormlearn;
```

```
USE ormlearn;
```

```
CREATE TABLE country (  
  co_code VARCHAR(2) PRIMARY KEY,  
  co_name VARCHAR(50)  
);
```

```
INSERT INTO country VALUES ('IN','India');  
INSERT INTO country VALUES ('US','United States of America');
```

Configure application.properties

```
# Spring Framework Log  
logging.level.org.springframework=info  
logging.level.com.cognizant=debug  
  
# Hibernate Log  
logging.level.org.hibernate.SQL=trace  
logging.level.org.hibernate.type.descriptor.sql=trace  
  
# Console Pattern  
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p %-25.25logger{25} %25M %4L  
%m%n  
  
# Database Configuration  
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn  
spring.datasource.username=root
```

```
spring.datasource.password=root
```

```
# Hibernate Configuration
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

Create Entity Class

Create package:

```
com.cognizant.ormlearn.model
```

Create **Country.java**

```
package com.cognizant.ormlearn.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "country")
public class Country {

    @Id
    @Column(name = "co_code")
    private String code;

    @Column(name = "co_name")
    private String name;

    public Country() {
    }

    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```

@Override
public String toString() {
    return "Country [code=" + code + ", name=" + name + "]\n";
}
}

```

Create Repository

Create package:

```
com.cognizant.ormlearn.repository
```

Create **CountryRepository.java**

```

package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {

}

```

Create Service Class

Create package:

```
com.cognizant.ormlearn.service
```

Create **CountryService.java**

```

package com.cognizant.ormlearn.service;

import java.util.List;

import javax.transaction.Transactional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional

```

```

    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}

```

Modify OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService;

    public static void main(String[] args) {

        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);

        countryService = context.getBean(CountryService.class);

        LOGGER.info("Inside main");

        testGetAllCountries();
    }

    private static void testGetAllCountries() {

        LOGGER.info("Start");

        List<Country> countries = countryService.getAllCountries();

        LOGGER.debug("Countries : {}", countries);

        LOGGER.info("End");
    }
}

```

Expected Output

Inside main

Start

Countries : [Country [code=IN, name=India],
Country [code=US, name=United States of America]]

End

Hands-on 2: Hibernate XML Configuration Implementation Walkthrough

Aim

To understand the implementation of Hibernate using XML configuration and perform database operations such as insert, retrieve, update, and delete using Hibernate.

Software Requirements

- Java 8 or above
- Eclipse IDE
- MySQL Server
- Hibernate Framework
- MySQL JDBC Driver

Introduction

Hibernate is an Object Relational Mapping (ORM) framework that simplifies database operations in Java applications. It maps Java objects to database tables using XML configuration files or annotations. In XML-based configuration, all database and mapping information is stored in XML files instead of Java annotations.

Hibernate XML Configuration Files

Hibernate uses the following XML files:

1. hibernate.cfg.xml

This file contains the database configuration details.

It includes:

- Database Driver
- Database URL
- Username
- Password
- Hibernate Dialect
- Mapping file information

Example:

```
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>

<session-factory>

<property name="connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>

<property name="connection.url">
jdbc:mysql://localhost:3306/hibernate
</property>

<property name="connection.username">
root
</property>

<property name="connection.password">
root
</property>

<property name="dialect">
org.hibernate.dialect.MySQL5Dialect
</property>

<mapping resource="Employee.hbm.xml"/>

</session-factory>

</hibernate-configuration>
```

2. Employee.hbm.xml

This file maps the Java class to the database table.

Example:

```
<hibernate-mapping>

<class name="Employee" table="employee">

<id name="id" column="id"/>

<property name="name" column="name"/>

<property name="salary" column="salary"/>

</class>

</hibernate-mapping>
```

Object Relational Mapping (ORM)

Object Relational Mapping (ORM) is the process of mapping Java objects to relational database tables.

Using ORM, developers work with Java objects instead of writing SQL queries directly.

Hibernate Architecture Components

1. SessionFactory

- SessionFactory is responsible for creating Session objects.
- It is created only once during application startup.
- It is a heavyweight object.

Example:

```
SessionFactory factory =  
new Configuration().configure().buildSessionFactory();
```

2. Session

- Session represents a connection between the application and the database.
- It is used to perform CRUD operations.
- It is created from the SessionFactory.

Example:

```
Session session = factory.openSession();
```

3. Transaction

- A transaction ensures that all database operations are completed successfully.
- If any operation fails, all changes can be rolled back.

Example:

```
Transaction tx = session.beginTransaction();
```

Hibernate Operations

beginTransaction()

- Starts a new transaction before performing database operations.

Example:

```
Transaction tx = session.beginTransaction();
```

commit()

- Saves all changes permanently into the database.

Example:

```
tx.commit();
```

rollback()

- Cancels all changes if an error occurs during execution.

Example:

```
tx.rollback();
```

session.save()

- Inserts a new object into the database.

Example:

```
Employee employee = new Employee();
```

```
employee.setId(101);  
employee.setName("John");  
employee.setSalary(50000);
```

```
session.save(employee);
```

session.createQuery().list()

- Retrieves all records from the database.

Example:

```
List<Employee> employees =  
session.createQuery("FROM Employee").list();
```

session.get()

- Retrieves a single object using its primary key.

Example:

```
Employee employee =
```



```
session.get(Employee.class,101);
```

session.delete()

- Deletes an object from the database.

Example:

```
session.delete(employee);
```

Advantages of Hibernate XML Configuration

- Provides object-relational mapping between Java objects and database tables.
- Eliminates the need for writing SQL queries for basic CRUD operations.
- Database-independent.
- Easy to maintain and modify configuration through XML files.
- Supports transaction management.
- Improves code readability and reusability.

Hands-on 3: Hibernate Annotation Configuration Implementation Walkthrough

Aim

To understand the implementation of Hibernate using annotations and perform object-relational mapping between Java classes and database tables without using XML mapping files.

Software Requirements

- Java 8 or above
- Eclipse IDE
- MySQL Server
- Hibernate Framework
- MySQL JDBC Driver

Persistence Class (Employee.java)

The persistence class represents the database table. Hibernate annotations are used to map the class and its fields to the corresponding table and columns.

Example:

```
import javax.persistence.Column;  
import javax.persistence.Entity;  
import javax.persistence.GeneratedValue;  
import javax.persistence.Id;  
import javax.persistence.Table;
```

```

@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue
    @Column(name = "id")
    private int id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "salary")
    private int salary;

    // Getters and Setters
}

```

Object Relational Mapping (ORM)

Object Relational Mapping (ORM) is the process of mapping Java objects to relational database tables.

Example:

Java Class Database Table

Employee	employee
id	id
firstName	first_name
lastName	last_name
salary	salary

ORM allows developers to work with Java objects instead of writing SQL statements manually.

Hibernate Annotations

1. @Entity

- Marks the class as a Hibernate entity.
- Indicates that the class is mapped to a database table.

Example:

```

@Entity

```

2. @Table

- Specifies the database table associated with the entity.

Example:

```
@Table(name = "employee")
```

3. @Id

- Specifies the primary key of the table.

Example:

```
@Id  
private int id;
```

4. @GeneratedValue

- Automatically generates the value of the primary key.

Example:

```
@GeneratedValue  
private int id;
```

5. @Column

- Maps a Java field to a specific database column.

Example:

```
@Column(name = "salary")  
private int salary;
```

Hibernate Configuration (hibernate.cfg.xml)

The hibernate.cfg.xml file contains the database configuration required to connect Hibernate with the database.

Example:

```
<!DOCTYPE hibernate-configuration PUBLIC  
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"  
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">  
  
<hibernate-configuration>  
  
<session-factory>
```

```
<property name="hibernate.dialect">
org.hibernate.dialect.MySQL5Dialect
</property>

<property name="hibernate.connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>

<property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/hibernate
</property>

<property name="hibernate.connection.username">
root
</property>

<property name="hibernate.connection.password">
root
</property>

<mapping class="Employee"/>

</session-factory>

</hibernate-configuration>
```

Hibernate Configuration Properties

1. Dialect

- Specifies the SQL dialect used by Hibernate.
- Helps Hibernate generate SQL compatible with the selected database.

Example:

```
<property name="hibernate.dialect">
org.hibernate.dialect.MySQL5Dialect
</property>
```

2. Driver

- Specifies the JDBC driver used to connect to the database.

Example:

```
<property name="hibernate.connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>
```

3. Connection URL

- Specifies the database URL.

Example:

```
<property name="hibernate.connection.url">  
jdbc:mysql://localhost:3306/hibernate  
</property>
```

4. Username

- Specifies the username used to connect to the database.

Example:

```
<property name="hibernate.connection.username">  
root  
</property>
```

5. Password

- Specifies the password used for database authentication.

Example:

```
<property name="hibernate.connection.password">  
root  
</property>
```

Advantages of Hibernate Annotation Configuration

- Reduces the need for separate XML mapping files.
- Keeps mapping information within the Java class.
- Easy to read and maintain.
- Simplifies development.
- Supports automatic table mapping.
- Provides better code readability and maintainability.

Hands-on 4: Difference between JPA, Hibernate and Spring Data JPA

Aim

To understand the differences between Java Persistence API (JPA), Hibernate, and Spring Data JPA, and compare their implementation for performing database operations.

Introduction

JPA, Hibernate, and Spring Data JPA are widely used technologies for interacting with relational databases in Java applications. Although they are closely related, each serves a different purpose. JPA defines the standard specification, Hibernate provides the implementation, and Spring Data JPA simplifies database operations by reducing boilerplate code.

Java Persistence API (JPA)

Java Persistence API (JPA) is a specification for managing relational data in Java applications.

Features

- JPA is defined under **JSR 338**.
- It provides a standard API for persisting, retrieving, updating, and deleting data.
- It is only a specification and does not contain any implementation.
- It requires an implementation such as Hibernate.
- It makes applications independent of a specific ORM framework.

Hibernate

Hibernate is an Object Relational Mapping (ORM) framework that implements the JPA specification.

Features

- Implements JPA interfaces.
- Maps Java objects to database tables.
- Provides automatic SQL generation.
- Supports transaction management.
- Reduces the need for manual JDBC programming.
- Offers additional features beyond JPA, such as caching and lazy loading.

Spring Data JPA

Spring Data JPA is a Spring framework module that works on top of JPA implementations like Hibernate.

Features

- Does not implement JPA by itself.
- Uses Hibernate (or another JPA provider) internally.
- Reduces boilerplate code.
- Automatically provides CRUD operations.
- Simplifies repository creation.
- Manages transactions using Spring's transaction management.

Difference between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Spring Module
Implementation	No	Yes	No
Purpose	Defines persistence standards	Implements JPA	Simplifies JPA usage
SQL Generation	No	Yes	Uses Hibernate
CRUD Operations	Defined by API	Implemented manually	Automatically available
Transaction Management	Specification only	Supported	Managed by Spring
Boilerplate Code	More	Moderate	Very Less

Hibernate Implementation Example

In Hibernate, database operations require manual creation of Session, Transaction, commit, rollback, and session closing.

```
public Integer addEmployee(Employee employee) {  
  
    Session session = factory.openSession();  
    Transaction tx = null;  
    Integer employeeID = null;  
  
    try {  
  
        tx = session.beginTransaction();  
  
        employeeID = (Integer) session.save(employee);  
  
        tx.commit();  
  
    } catch (HibernateException e) {
```

```

        if (tx != null)
            tx.rollback();

        e.printStackTrace();

    } finally {

        session.close();

    }

    return employeeID;
}

```

Explanation

- Opens a Hibernate session.
- Begins a transaction.
- Saves the employee object.
- Commits the transaction if successful.
- Rolls back the transaction if an exception occurs.
- Closes the session.

Spring Data JPA Implementation Example

EmployeeRepository.java

```

import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

}

```

EmployeeService.java

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import javax.transaction.Transactional;

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional
    public void addEmployee(Employee employee) {

        employeeRepository.save(employee);
    }
}

```



```
}  
}
```

Explanation

- Extends JpaRepository.
- No need to create Session or Transaction manually.
- save() performs the insert operation.
- @Transactional automatically manages the transaction.
- Reduces the amount of code required.

Comparison of Hibernate and Spring Data JPA

Hibernate	Spring Data JPA
Manual session management	Automatic session management
Manual transaction handling	Automatic transaction handling
More code required	Less code required
Manual CRUD implementation	Built-in CRUD methods
Direct ORM framework	Uses Hibernate internally

Advantages

JPA

- Standard persistence specification.
- Portable across different ORM frameworks.
- Database independent.

Hibernate

- Powerful ORM framework.
- Automatic SQL generation.
- Supports caching and lazy loading.
- Reduces JDBC coding.

Spring Data JPA

- Simplifies data access.
- Reduces boilerplate code.
- Built-in CRUD methods.
- Easy repository creation.
- Automatic transaction management.

Hands-on 5: Implement Services for Managing Country

Aim

To configure Hibernate table creation settings, populate the country table with data, and prepare the Spring Data JPA application for implementing Country management services.

Introduction

Before implementing Country management services, the database must be configured properly and populated with country records. Hibernate provides the `spring.jpa.hibernate.ddl-auto` property to control how the database schema is managed during application startup. The country table will be populated with country details, which will be used in the subsequent hands-on exercises to perform CRUD operations.

Hibernate Table Creation Configuration

Add the following property in `application.properties`.

```
spring.jpa.hibernate.ddl-auto=validate
```

The `ddl-auto` property specifies how Hibernate manages the database schema.

Hibernate ddl-auto Options

1. create

- Deletes existing tables.
- Creates new tables every time the application starts.
- Existing data will be lost.

Example:

```
spring.jpa.hibernate.ddl-auto=create
```

2. validate

- Validates whether the database tables and columns exist.
- Does not modify the database.
- Throws an exception if the schema is incorrect.

Example:

```
spring.jpa.hibernate.ddl-auto=validate
```

3. update

- Updates the existing database schema.
- Creates missing tables or columns if required.
- Preserves existing data.

Example:

```
spring.jpa.hibernate.ddl-auto=update
```

4. create-drop

- Creates tables when the application starts.
- Automatically drops the tables when the application stops.

Example:

```
spring.jpa.hibernate.ddl-auto=create-drop
```

Comparison of ddl-auto Options

Option	Purpose
create	Creates new tables after deleting existing ones.
validate	Validates the schema without making changes.
update	Updates the schema while preserving data.
create-drop	Creates tables at startup and drops them at shutdown.

Database Population

Populate the country table with the country records provided in the SQL script.

Sample SQL commands:

```
DELETE FROM country;
```

```
INSERT INTO country (co_code, co_name) VALUES ('IN', 'India');
INSERT INTO country (co_code, co_name) VALUES ('US', 'United States');
INSERT INTO country (co_code, co_name) VALUES ('GB', 'United Kingdom');
INSERT INTO country (co_code, co_name) VALUES ('JP', 'Japan');
INSERT INTO country (co_code, co_name) VALUES ('AU', 'Australia');
```

Note: Execute the complete SQL script provided in the hands-on document to populate the country table with all country records.

Services to be Implemented

After populating the database, the following Country management services will be implemented in the subsequent hands-on exercises:

1. Find Country by Code

Retrieve a country using its unique country code.

2. Add Country

Insert a new country into the database.

3. Update Country

Modify the details of an existing country.

4. Delete Country

Remove a country from the database using its country code.

5. Find Countries by Partial Name

Retrieve all countries whose names match a given partial string.

Advantages

- Simplifies database schema management.
- Validates the database structure before application startup.
- Supports automatic schema creation and updates.
- Provides a ready-to-use database for CRUD operations.
- Makes it easier to implement Country management services.

Expected Outcome

- Hibernate successfully validates the database schema.
- The country table is populated with all country records.
- The application is ready for implementing Country management services such as Find, Add, Update, Delete, and Search.

Hands-on 6: Find a Country Based on Country Code

Aim

To retrieve a country from the database using its country code by implementing the `findCountryByCode()` method in the service layer using Spring Data JPA.

Software Requirements

- Java 8 or above
- Spring Boot
- Spring Data JPA
- MySQL Server
- Eclipse IDE
- Maven

Introduction

Spring Data JPA provides the built-in `findById()` method to retrieve a record using its primary key. In this hands-on, a service method is implemented to fetch a country based on its country code. If the specified country code does not exist in the database, a custom exception is thrown.

Create Custom Exception

Create the package:

```
com.cognizant.ormlearn.service.exception
```

Create the class **CountryNotFoundException.java**

```
package com.cognizant.ormlearn.service.exception;

public class CountryNotFoundException extends Exception {

    public CountryNotFoundException(String message) {
        super(message);
    }
}
```

Implement `findCountryByCode()` in **CountryService**

Add the following method in **CountryService.java**

```
package com.cognizant.ormlearn.service;

import java.util.Optional;
```

```

import javax.transaction.Transactional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public Country findCountryByCode(String countryCode)
        throws CountryNotFoundException {

        Optional<Country> result =
            countryRepository.findById(countryCode);

        if (!result.isPresent()) {
            throw new CountryNotFoundException(
                "Country not found with code : " + countryCode);
        }

        Country country = result.get();

        return country;
    }
}

```

Create Test Method

Add the following method in **OrmLearnApplication.java**

```

private static void testFindCountryByCode()
    throws CountryNotFoundException {

    LOGGER.info("Start");

    Country country =
        countryService.findCountryByCode("IN");

    LOGGER.debug("Country : {}", country);
}

```

```
    LOGGER.info("End");  
}
```

Invoke the Test Method

Modify the main() method.

```
ApplicationContext context =  
    SpringApplication.run(OrmLearnApplication.class, args);  
  
countryService = context.getBean(CountryService.class);  
  
testFindCountryByCode();
```

Importance of @Transactional

The @Transactional annotation is used to manage database transactions automatically.

Advantages

- Automatically begins a transaction.
- Commits the transaction after successful execution.
- Rolls back the transaction if an exception occurs.
- Eliminates the need for manual transaction management.
- Spring manages the Hibernate session automatically.

Expected Output

If the country code exists:

Start

Country : Country [code=IN, name=India]

End

If the country code does not exist:

CountryNotFoundException:

Hands-on 7: Add a New Country

Aim

To add a new country record into the database using the `save()` method provided by Spring Data JPA.

Software Requirements

- Java 8 or above
- Spring Boot
- Spring Data JPA
- MySQL Server
- Eclipse IDE
- Maven

Introduction

Spring Data JPA provides the built-in `save()` method to insert new records into the database. In this hands-on, a new country object is created and saved into the country table. After insertion, the record is retrieved using its country code to verify that it has been successfully added.

Step 1: Implement `addCountry()` in `CountryService`

Add the following method in **CountryService.java**

```
@Transactional
public void addCountry(Country country) {

    countryRepository.save(country);

}
```

Step 2: Create Test Method

Add the following method in **OrmLearnApplication.java**

```
private static void testAddCountry()
    throws CountryNotFoundException {

    LOGGER.info("Start");

    Country country = new Country();

    country.setCode("NP");
    country.setName("Nepal");
```



```

countryService.addCountry(country);

Country result =
    countryService.findCountryByCode("NP");

LOGGER.debug("Country : {}", result);

LOGGER.info("End");
}

```

Step 3: Invoke the Test Method

Modify the main() method.

```

ApplicationContext context =
    SpringApplication.run(OrmLearnApplication.class, args);

countryService = context.getBean(CountryService.class);

testAddCountry();

```

Verification

After executing the application:

1. Open **MySQL Workbench**.
2. Execute the following query:

```

SELECT * FROM country
WHERE co_code='NP';

```

The newly added country should be displayed.

Expected Output

Console Output

Start

Country : Country [code=NP, name=Nepal]

End

Database Output

Country Code Country Name

Country Code Country Name

NP Nepal

Advantages

- Uses the built-in save() method provided by Spring Data JPA.
- No SQL INSERT statement is required.
- Automatically manages transactions using @Transactional.
- Simplifies database insertion with minimal code.

Hands-on 8: Update a Country Based on Code

Aim

To update the name of an existing country in the database using its country code with the help of Spring Data JPA.

Software Requirements

- Java 8 or above
- Spring Boot
- Spring Data JPA
- MySQL Server
- Eclipse IDE
- Maven

Introduction

Spring Data JPA provides the `findById()` and `save()` methods to retrieve and update records in a database. In this hands-on, an existing country is retrieved using its country code, its name is modified, and the updated record is saved back to the database.

Implement `updateCountry()` in `CountryService`

Add the following method in **CountryService.java**

```
@Transactional
public void updateCountry(String code, String name)
    throws CountryNotFoundException {

    Optional<Country> result =
        countryRepository.findById(code);

    if (!result.isPresent()) {
        throw new CountryNotFoundException(
            "Country not found with code : " + code);
    }

    Country country = result.get();

    country.setName(name);

    countryRepository.save(country);
}
```

Create Test Method

Add the following method in **OrmLearnApplication.java**

```
private static void testUpdateCountry()
    throws CountryNotFoundException {

    LOGGER.info("Start");

    countryService.updateCountry(
        "NP",
        "Federal Democratic Republic of Nepal");

    Country country =
        countryService.findCountryByCode("NP");

    LOGGER.debug("Updated Country : {}", country);

    LOGGER.info("End");
}
```

Invoke the Test Method

Modify the main() method.

```
ApplicationContext context =
    SpringApplication.run(OrmLearnApplication.class, args);

countryService = context.getBean(CountryService.class);

testUpdateCountry();
```

Verification

Execute the following SQL query in MySQL Workbench:

```
SELECT * FROM country
WHERE co_code='NP';
```

The updated country name should be displayed.

Expected Output

Console Output

Start

Updated Country : Country [code=NP,
name=Federal Democratic Republic of Nepal]

End

Database Output

Country Code	Country Name
NP	Federal Democratic Republic of Nepal

Advantages

- Uses built-in Spring Data JPA methods.
- Simplifies update operations.
- No SQL UPDATE statement is required.
- Transaction management is handled automatically using `@Transactional`.
- Improves code readability and maintainability.

Hands-on 9: Delete a Country Based on Code

Aim

To delete an existing country from the database using its country code with the help of Spring Data JPA.

Software Requirements

- Java 8 or above
- Spring Boot
- Spring Data JPA
- MySQL Server
- Eclipse IDE
- Maven

Introduction

Spring Data JPA provides the built-in `deleteById()` method to delete records from the database using the primary key. In this hands-on, a country is deleted using its country code, and the deletion is verified by checking the database.

Step 1: Implement `deleteCountry()` in `CountryService`

Add the following method in **CountryService.java**

```
@Transactional
public void deleteCountry(String countryCode) {

    countryRepository.deleteById(countryCode);
```

```
}
```

Create Test Method

Add the following method in **OrmLearnApplication.java**

```
private static void testDeleteCountry() {  
  
    LOGGER.info("Start");  
  
    countryService.deleteCountry("NP");  
  
    LOGGER.info("Country deleted successfully.");  
  
    LOGGER.info("End");  
}
```

Invoke the Test Method

Modify the main() method.

```
ApplicationContext context =  
    SpringApplication.run(OrmLearnApplication.class, args);  
  
countryService = context.getBean(CountryService.class);  
  
testDeleteCountry();
```

Verification

After executing the application, open **MySQL Workbench** and execute the following query:

```
SELECT * FROM country  
WHERE co_code='NP';
```

If the deletion is successful, no records will be returned.

Expected Output

Console Output

Start

Country deleted successfully.

End

Database Output

Empty Set

or

0 rows returned

This confirms that the country has been deleted successfully.

Advantages

- Uses the built-in deleteById() method provided by Spring Data JPA.
- Eliminates the need to write SQL DELETE statements manually.
- Automatically manages transactions using @Transactional.
- Simplifies database delete operations with minimal code.
- Improves code readability and maintainability.